NIRVAAN: A Blockchain-Based E-Tendering Platform for Oil PSUs

Course Project Final Submission | BACSE350 - Blockchain Architecture and Design

Tharun Revinth V S (Reg No: 25BCE5268)

October 2026

1. Project Title

Project Title: NIRVAAN: A Blockchain-Based E-Tendering Platform for Oil PSUs
Course Code: BACSE350
Course Name: Blockchain Architecture and Design
Faculty Guide: Prof. Malathi G
Academic Term: Fall Semester 2026
Due Date: 29 October 2026, 11:59 PM
Submission Date: October 2026

2. Team Members with Name and Registration Number

This project is an individual course capstone project developed, evaluated, and submitted by:

S. No.	Student Name	Registration Number	Role in Project	Contribution Scope
1	**Tharun Revinth V S**	**25BCE5268**	Sole Developer / Lead Researcher	Smart contracts design and testing, Express backend relays and indexing, React frontend UI integration, cloud deployment, and system verification.

(Note: Additional rows may be added by the student if team composition is adjusted prior to final submission.)

3. Problem Definition

Public procurement across India's state-owned oil and gas Public Sector Undertakings (PSUs)—including Indian Oil Corporation Limited (IOCL), Bharat Petroleum Corporation Limited (BPCL), Hindustan Petroleum Corporation Limited (HPCL), Oil and Natural Gas Corporation (ONGC), and Gas Authority of India Limited (GAIL)—involves high-value engineering, procurement, and construction (EPC) contracts. These encompass specialized refining components, offshore drilling machinery, and cross-country pipeline infrastructure.

Although centralized electronic procurement systems such as the Central Public Procurement Portal (CPPP) and Government e-Marketplace (GeM) have replaced physical paper envelopes, their underlying operational model relies on centralized database management systems (RDBMS) administered by single organizations. This centralized architecture exhibits several systemic vulnerabilities:

1. **Premature Commercial Bid Leakage:** In competitive tendering, early disclosure of commercial quotes compromises the entire bidding process. In centralized systems, financial proposals are uploaded to a central server. Even when encrypted, database administrators (DBAs) or compromised insiders holding cryptographic decryption keys can inspect commercial bids prior to the scheduled unsealing date. Privileged contractors can be informed of competing bids in advance, allowing them to undercut legitimate competitors by fractional margins.
2. **Bid Tampering and Retroactive Modifications:** Standard relational databases operate on mutable primitives (`UPDATE`, `DELETE`). A privileged system administrator or attacker with root credentials can modify proposal values, adjust recorded submission timestamps, or replace uploaded technical documents without generating an immutable audit trail.
3. **Weak and Scrubbable Audit Trails:** Centralized server logs are stored on the same physical or cloud hosting environments as the application. Corrupt actors can retroactively truncate or falsify application and database logs, eliminating evidence of administrative tampering.
4. **Vendor Collusion and Bidder Cartels:** In specialized energy procurement with limited domestic suppliers, informal vendor cartels can coordinate bids to artificially inflate prices while alternating wins across tenders. Centralized systems lack transparent, cross-PSU historical bidding registries to verify supplier behavior.
5. **Subjective Disqualification and Disputed Awards:** During technical evaluation, procurement committees may subjectively assign lower scores (e.g., 68/100 against a 70/100 cutoff) to disqualify honest lower-priced contractors, subsequently awarding contracts to favored, more expensive contractors (L2 or L3). Centralized portals lack on-chain cryptographic enforcement linking technical scoring to commercial award rules.
6. **Delayed Earnest Money Deposit (EMD) Refunds:** Under General Financial Rules (GFR 2017) Rule 170, participating bidders must deposit bid security (EMD) to discourage frivolous bidding. In centralized systems, processing EMD refunds for non-winning bidders requires manual administrative clearances and banking transfers, routinely locking up vendor capital for months.
7. **Over-Trust in a Single Administrative Authority:** Centralized systems force all external bidders to place absolute trust in the contracting entity that issues the tender, evaluates the proposals, and holds the funds. When disputes occur, suppliers must appeal to the very organization accused of bias.

Stakeholders

The NIRVAAN platform interfaces five primary classes of stakeholders: - **PSU Procurement Officers:** Authorized officials who issue Notices Inviting Tender (NIT), set deadlines, review vendor onboarding documents, score technical proposals, and initiate contract awards. - **Commercial Vendors:** Verified domestic and international industrial contractors who complete Know-Your-Customer (KYC) onboarding, submit sealed bids backed by EMD escrow, reveal their quotations during the reveal window, and withdraw refunded EMD capital. - **Central Vigilance Commission (CVC) Auditors:** Independent statutory oversight officers with on-chain authority to audit the complete custody timeline, review grievances filed during the statutory standstill window, and formally uphold or dismiss disputes. - **Autonomous Keepers:** Algorithmic automation daemons monitoring blockchain timestamps to trigger scheduled state transitions (such as unsealing bids and finalizing contracts) without human discretion. - **Consortium / System Admin:** Administrative multisig authority responsible for managing registered PSU accounts and authorizing UUPS contract implementation upgrades.

Objectives

1. Implement a **non-interactive commit-reveal cryptographic bidding mechanism** that mathematically prevents quotation leakage until the submission deadline expires.
2. Enforce the statutory **Lowest Evaluated Responsive Bidder (L1) award rule** directly on-chain within smart contract logic, prohibiting arbitrary awards to higher-priced vendors.
3. Automate **EMD escrow releases via non-custodial pull-payments**, enabling non-winning bidders to withdraw their funds immediately upon tender finalization.
4. Encode a **statutory 7-day standstill governance window** post-award, permitting disqualified bidders to lodge formal grievances arbitrated by an independent CVC role with a 14-day inactivity timeout.
5. Deploy a full-stack, transparent prototype on the Ethereum Sepolia testnet integrated with decentralized document storage (IPFS).

Scope and Out-of-Scope

- **In-Scope:** End-to-end tender lifecycle: vendor KYC application and officer verification; tender notice creation with IPFS specification hashing; sealed bid submission with client-side 32-byte salt generation; keeper-automated bid opening; cryptographic bid reveal; technical proposal evaluation via backend gas relays; on-chain L1 award verification; standstill dispute filing and CVC resolution; and pull-payment EMD withdrawals.
- **Out-of-Scope:** Real-time fiat currency banking rails (INR payments are modeled in Sepolia testnet ETH); zero-knowledge proof generation for complex technical score calculations; and high-volume retail catalog procurement (which is already served by GeM).

---

4. Need for Blockchain

A standard relational database (e.g., PostgreSQL or MySQL) is insufficient for public procurement because database operations rely entirely on administrative trust. A database administrator with root credentials can alter state tables, backdate timestamps, modify audit logs, or leak confidential data.

In contrast, blockchain technology replaces institutional trust with cryptographic consensus. Table 1 outlines how specific blockchain properties directly address the core failure modes of centralized e-tendering:

Procurement Problem	Centralized Database Limitation	Blockchain Cryptographic Property	How NIRVAAN Solves the Problem
Premature Bid Leakage	Database admin or KMS holder can decrypt financial proposals prior to tender opening.	**Cryptographic Commit-Reveal (`keccak256`)**	The bid quote is hashed on the client with a 32-byte salt. Only the hash is stored on-chain. Quotes cannot be computed from the hash prior to reveal.
Post-Submission Tampering	Records can be altered via SQL `UPDATE` or `DELETE` statements.	**Append-Only Immutability**	Once recorded inside an EVM block, transaction calldata and state storage cannot be altered or deleted by any entity.
Subjective L1 Bypass	Officers can manually select a higher-priced vendor in the application UI.	**Deterministic Smart Contract Execution**	The `awardTender` function iterates over all opened bids on-chain and reverts if the proposed winner is not the lowest qualified bid.
Weak Audit Trail	Server logs can be wiped, truncated, or edited by root users.	**Decentralized Event Logs & Block Receipts**	Every lifecycle event (`BidSubmitted`, `BidRevealed`, `TenderAwarded`) emits an EVM log indexed across all nodes, permanently visible on Etherscan.
Delayed EMD Refunds	Unsuccessful vendor funds are locked pending manual administrative processing.	**Programmable Pull-Payment Escrow**	The `finalizeTender` function automatically credits refundable EMD balances to vendor accounts; vendors withdraw instantly via `withdraw()`.
Arbitrary Disqualification	Rejected vendors have no independent appeal mechanism without court litigation.	**On-Chain Standstill & Role-Based CVC Arbitration**	Contracts enforce a mandatory 7-day standstill after award. A designated CVC auditor address has sole authority to uphold or dismiss disputes.

Deployment Reality & Honesty Statement

NIRVAAN is deployed as a functional **research prototype on the public Ethereum Sepolia testnet**. In an actual enterprise production deployment across Indian oil PSUs, the platform would be hosted on a private, permissioned network (such as Hyperledger Besu utilizing the QBFT consensus algorithm).

A permissioned network would eliminate public transaction fees, provide sub-second transaction finality, and restrict validator consensus strictly to authorized state institutions (IOCL, BPCL, HPCL, ONGC, GAIL, and the Ministry of Petroleum and Natural Gas). For this academic demonstration, the system runs on the public Sepolia testnet, and must **not** be referred to as a private consortium blockchain in its current deployed form.

---

5. YouTube Upload Link

YouTube link: [TO BE ADDED BY STUDENT AFTER UPLOAD]

(Note: The student should record a concise 5 to 10 minute video demonstration showing vendor KYC application, officer on-chain approval, tender creation, sealed bid submission, bid reveal, technical scoring, L1 award, and EMD withdrawal, then paste the public URL above.)

---

6. Project Launch Link

The NIRVAAN prototype is fully deployed and accessible via the following public endpoints:

- **Live Application Frontend:** https://nirvaan-flame.vercel.app/
  (Hosted on Vercel Global Edge Network with continuous deployment from the GitHub master branch.)
- **Backend Health Check & Indexer Status:** https://nirvaan-backend-84fc.onrender.com/api/health
  (Hosted on Render Web Service running Node.js 22 LTS.)
- **Deployed ERC-1967 Proxy Contract on Sepolia:**
  `0xe9649E82dC4fEdf86dFbE5120a3fE79BDde88663`
  Etherscan Link: https://sepolia.etherscan.io/address/0xe9649E82dC4fEdf86dFbE5120a3fE79BDde88663
- **Deployed Logic Implementation Contract (`OilTenderingV1`):**
  `0xd5451fe17e92B9A9A8Abce4C6c1fc5bFF86f6606`
  Deployment Block Height: `11959326` | *Network Chain ID:* `11155111` (Sepolia PoS)

> ***Operational Note on Free Cloud Hosting:*** *The backend service is hosted on Render's free tier. If the service has been inactive, the container spins down automatically; initial requests to the API or health check endpoint may take up to 60 seconds to wake up. Demonstration credentials for PSU officers and test vendor accounts are provided separately to evaluators. In accordance with security practices, no private keys or account passwords are printed in this document.*

---

7. Softwares Used

Table 2 lists all software frameworks, runtimes, protocols, and developer tools used across the NIRVAAN codebase, along with exact version numbers extracted from configuration files:

Software / Tool	Exact Version	Primary Purpose and Functional Scope
Solidity	`^0.8.24`	Smart contract programming language; compiled with default checked arithmetic to eliminate overflow/underflow vulnerabilities.
OpenZeppelin Contracts	`5.0.2`	Audited contract implementations: `Initializable` and `UUPSUpgradeable` (`ERC-1967` proxy pattern).
Hardhat	`2.22.0`	Local Ethereum development suite, test runner, gas profiling, and contract compilation environment.
ethers.js	`v6.17.0` (backend) / `v6.13.2` (frontend)	Complete Web3 library for transaction signing, calldata encoding, contract event listening, and BigInt arithmetic.
Node.js	`v24.17.0` (LTS `>=22.0.0`)	JavaScript server runtime powering the Express API server, event indexer, and embedded keeper bot.
Express.js	`4.19.2`	REST API routing framework managing officer authentication, file upload challenges, and gas relays.
SQLite (`node:sqlite`)	Built-in Node 22 Engine	Embedded, zero-dependency relational cache storing indexed on-chain events and pending KYC applications.
React	`18.3.1`	Declarative component UI library for the single-page application (SPA).
Vite	`5.4.1`	Next-generation frontend bundler providing fast build optimization and client-side environment injection.
Tailwind CSS	`3.4.10`	Utility-first CSS framework implementing responsive layouts and GIGW accessibility styling.
MetaMask	Extension (EIP-1193)	Client-side cryptographic wallet provider enabling vendors to sign challenge nonces and on-chain transactions.
Alchemy	JSON-RPC API	High-availability Ethereum Sepolia node infrastructure provider for querying blocks and submitting transactions.

Software / Tool	Exact Version	Primary Purpose and Functional Scope
Pinata	Cloud IPFS API / SDK	Content-addressed storage service pinning tender notices, engineering blueprints, and vendor KYC dossiers.
Vercel	Edge Network	Global serverless CDN hosting the production React frontend with automatic SSL/TLS termination.
Render	Cloud PaaS	Linux container hosting the Node.js Express API server, background event indexer, and keeper cron.
UptimeRobot	Monitoring Service	Synthetic HTTP ping monitor dispatching automated requests every 5 minutes to prevent backend container sleeping.
Git & GitHub	Version 2.x	Distributed version control repository and automated CI smoke testing environment.
VS Code	IDE	Primary development environment used for smart contract authoring, testing, and system integration.
Mermaid & Chrome	Mermaid 10.9.8 / Chrome 155	Diagramming specification language and headless browser engine used to render vector architecture diagrams and compile PDFs.

---

8. AI Models Used

In accordance with academic integrity guidelines, the use of generative artificial intelligence during the development of this project is fully disclosed:

1. **Claude (Anthropic - Claude 3.5 Sonnet / Claude 3 Opus):** Used as an interactive pair-programming assistant for:
   - Reviewing smart contract access-control logic and edge-case boundary conditions;
   - Debugging asynchronous event-indexing errors related to Alchemy’s 10-block query limits;
   - Providing configuration guidance for multi-tier cloud deployment across Render, Vercel, and Pinata; and
   - Structuring formal academic documentation and test checklists.
2. **AGY (Advanced Coding Assistant):** The primary AI coding agent used within the developer environment to generate code diffs, run test suites, execute gas profiling scripts, and refactor UI

components. *(Underlying Model: [STUDENT TO CONFIRM MODEL NAME, e.g., Claude 3.5 Sonnet / Gemini Pro]).*

Student Responsibility Statement: All AI-suggested code, configuration parameters, and architectural drafts were critically reviewed, manually tested, modified, and verified by the student (**Tharun Revinth V S**). The student holds full personal and academic responsibility for the functionality, security, test suite execution, and content of this submission.

---

9. Detailed Architecture Diagram

The NIRVAAN platform utilizes a decoupled four-tier architecture designed to enforce cryptographic invariants on-chain while maintaining high query performance through an off-chain indexing layer.

System Architecture Overview

Figure 1 details the full system topography, showing how end-user browsers, the Vercel edge network, the Render backend, Pinata IPFS, and the Ethereum Sepolia network communicate:

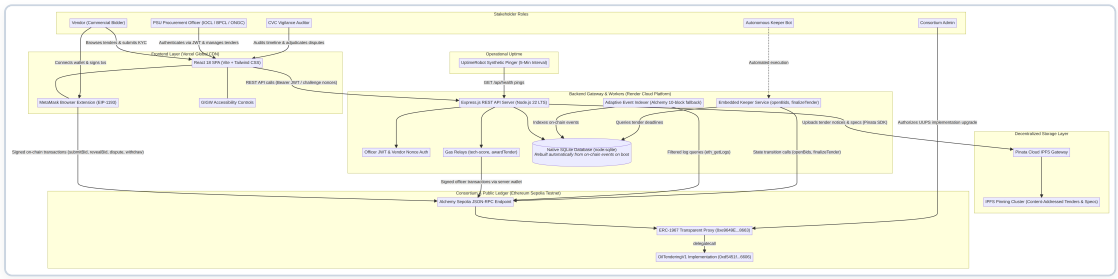


Figure 1: Comprehensive Multi-Tier System Architecture

Explanation of Figure 1: - **Presentation Layer (Vercel):** The React 18 single-page application is hosted globally on Vercel. Vendors interact with the system using MetaMask (EIP-1193) for cryptographic transaction signing and challenge-response authentication. Procurement officers access the dashboard using JWT credentials. The interface incorporates Guidelines for Indian Government Websites (GIGW) accessibility controls (font resizing and contrast toggles). - **Application & Worker Layer (Render):** The Express API server runs on Node.js 22 LTS. It enforces per-PSU tenant isolation (`req.officer.psu`), signs backend gas relays (`tech-score` and `awardTender`) using the server wallet, and manages cryptographic nonces for file uploads. The backend contains an embedded **Event Indexer** that continuously scans Sepolia blocks via `eth_getLogs` (with automatic 10-block fallback handling) to update an embedded **SQLite database**. An embedded **Keeper Bot** monitors block timestamps to automatically execute scheduled openings and finalizations. - **Decentralized Storage Layer (Pinata IPFS):** Large binary documents (tender specifications, engineering drawings, vendor registration certificates) are pinned to IPFS via Pinata. Only the resulting 46-character Content Identifiers (CIDs) are recorded on-chain, preventing excessive blockchain storage costs. - **Consortium & Public Ledger Layer (Sepolia):** State transitions are executed through an ERC-1967 transparent upgradeable proxy delegating calls to `OilTenderingV1`. All state mutations, bid commitments, EMD escrow balances, and dispute records reside in verified EVM storage.

Tender State Machine Diagram

Figure 2 models the lifecycle of a tender through its six finite states, including programmatic transition guards and access control boundaries:

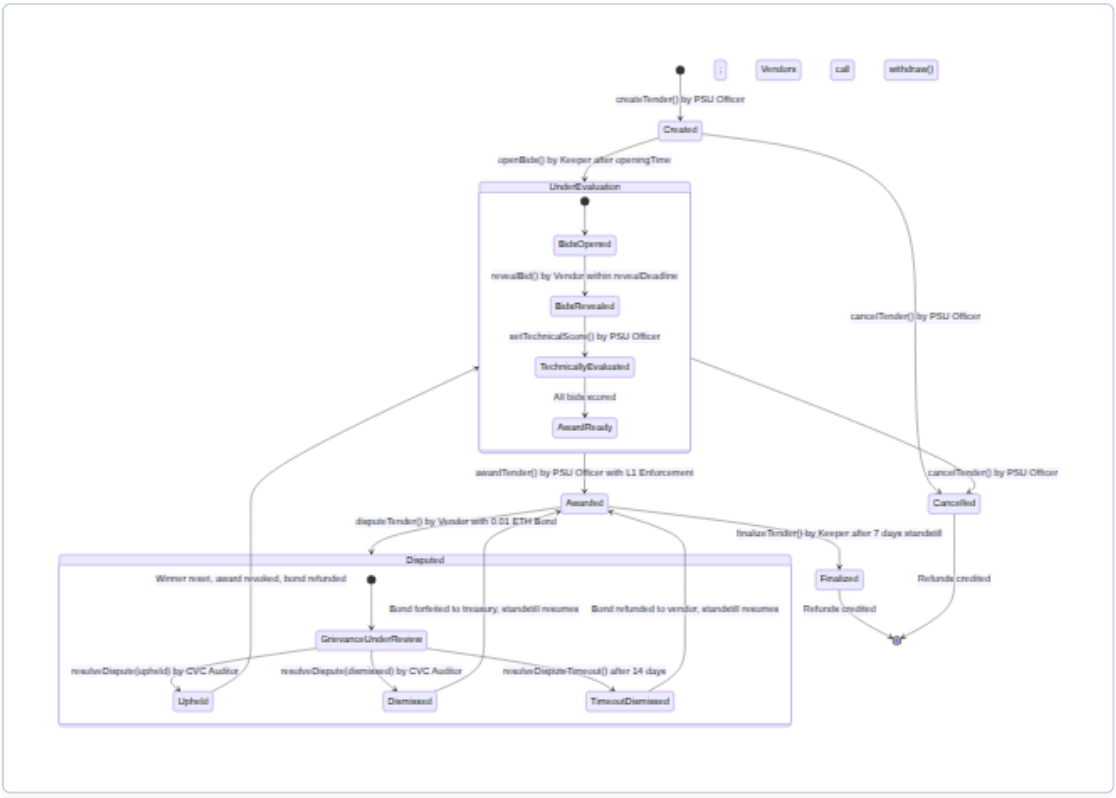


Figure 2: Tender Lifecycle Finite State Machine

Explanation of Figure 2: 1. **`Created`:** The tender is published by an authorized PSU officer with verified deadlines (now < subDeadline ≤ openTime < revDeadline). Verified vendors submit sealed bids accompanied by EMD deposits. 2. **`UnderEvaluation`:** Once `block.timestamp >= openingTime`, the autonomous keeper triggers `openBids()`. During the reveal window, vendors unseal their bids (`revealBid()`). Officers assign technical qualification scores (0–100) via backend gas relays. 3. **`Awarded`:** After the reveal deadline, the officer triggers `awardTender()`. The contract algorithmically verifies that the selected winner is technically qualified and represents the strictly lowest commercial quote (L1). A mandatory 7-day standstill appeal window begins. 4. **`Disputed`:** If an unsuccessful bidder appeals the award within the 7-day window by depositing a 0.01 ETH dispute bond, the tender enters the `Disputed` state. The CVC Auditor independently investigates the grievance: - *If Upheld:* The award is revoked, winner reset to `address(0)`, status returned to `UnderEvaluation`, and the bond is refunded. - *If Dismissed:* The bond is forfeited to the government treasury, and the status returns to `Awarded`. - *Failsafe Timeout:* If the CVC is inactive for >14 days, any caller can invoke `resolveDisputeTimeout()` to auto-resolve the standstill. 5. **`Finalized`:** Once 7 days elapse without an active dispute, the keeper calls `finalizeTender()`. The contract executes `_creditRefunds()`, crediting EMD refunds to all qualified bidders and the winner. Bidders withdraw funds via `withdraw()`. 6. **`Cancelled`:** If operational needs change prior to award, the issuing PSU can cancel the tender (`cancelTender()`), triggering automatic EMD refunds.

End-to-End Bid Lifecycle Sequence Diagram

Figure 3 captures the sequence of transactions and messages exchanged among the Bidder, PSU Officer, CVC Auditor, Backend Relay, Smart Contract, and Autonomous Keeper:

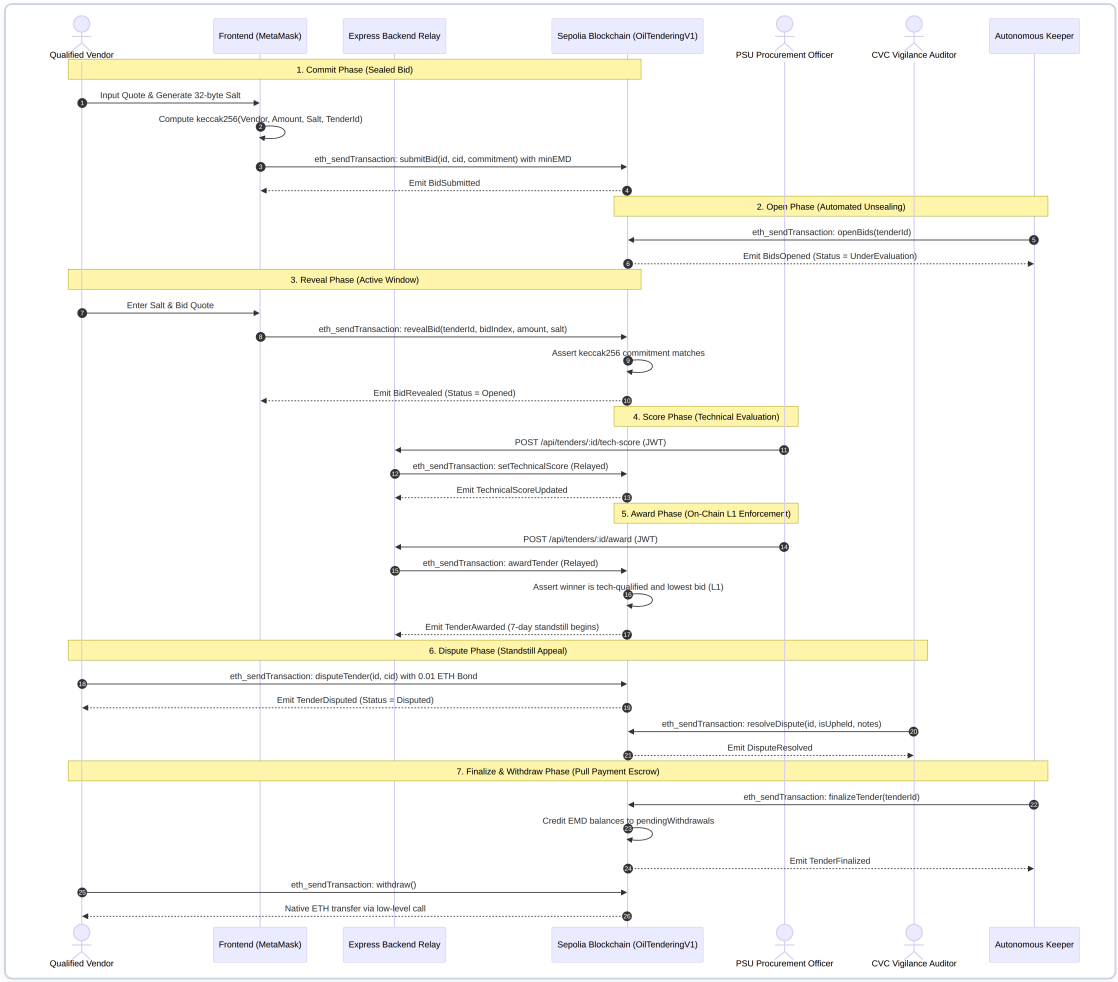


Figure 3: Bid Lifecycle Sequence Model

Explanation of Figure 3: 1. **Commit Phase:** The vendor inputs their commercial quote and generates a local 32-byte salt. The browser computes H = keccak256(msg.sender, bidAmount, salt, tenderId). The vendor submits H to `submitBid()` along with the required EMD value in Sepolia ETH. 2. **Open Phase:** After the submission deadline lapses, the autonomous keeper detects eligibility and calls `openBids()`, transitioning the status to `UnderEvaluation`. 3. **Reveal Phase:** During the reveal window, the vendor submits their quote and salt to `revealBid()`. The smart contract recomputes the hash and asserts equality against the stored commitment. 4. **Score Phase:** The PSU officer evaluates the vendor's technical documentation and submits scores via the authenticated `/api/tenders/:id/tech-score` backend relay. 5. **Award Phase:** After the reveal deadline closes, the officer triggers `/api/tenders/:id/award`. The contract executes the L1 invariant check: verifying the winner is technically qualified and holds the lowest quote. 6. **Dispute Phase:** An appealing vendor lodges a grievance with a 0.01 ETH bond. The CVC Auditor adjudicates on-chain. 7. **Finalize & Withdraw Phase:** The keeper finalizes the tender after the standstill expires. Bidders call `withdraw()` to claim their refunded deposits via pull payments.

10. Components of Blockchain and Explanation of the Smart Contract Used

Blockchain Components Used in NIRVAAN

The platform utilizes standard Ethereum architecture primitives: - **Network:** Ethereum Sepolia Testnet (PoS consensus, Chain ID: `11155111`). Provides an immutable public environment with 12-second block intervals. - **Accounts & Keypairs:** Secp256k1 elliptic curve keypairs. Vendors authenticate and sign transactions via MetaMask; the backend relay server signs officer operations using an institutional server wallet. - **Transactions & Calldata:** State-modifying operations (such as `submitBid`, `revealBid`, `awardTender`) are packaged into cryptographically signed transactions broadcasting EVM-encoded calldata. - **Blocks & Timestamps:** Ethereum block timestamps (`block.timestamp`) govern tender deadlines (submission, opening, reveal, and standstill appeal expiration). - **Gas & Execution Fees:** EVM execution is metered in gas units. Testnet gas is paid in Sepolia ETH. Contracts are optimized to ensure routine vendor operations consume $< 300,000$ gas. - **Events & Logs:** Contracts emit structured Solidity events (`TenderCreated`, `BidSubmitted`, `BidRevealed`, `TenderAwarded`, `DisputeResolved`). These are indexed by the backend using `eth_getLogs` to maintain an off-chain SQLite read replica. - **Consensus:** Sepolia uses Ethereum's Proof of Stake (PoS) consensus, providing distributed finality across international validator nodes. - **Smart Contracts & Storage Layout:** Logic is written in Solidity 0.8.24 and compiled into EVM bytecode. Storage variables are laid out according to ERC-1967 storage standards. - **Proxy Architecture:** Utilizes the Universal Upgradeable Proxy Standard (UUPS / ERC-1822) with an `ERC1967Proxy`. - **IPFS & Cryptographic Hashing:** High-volume documents are hashed using SHA-256 (IPFS CIDs). Commercial bids are committed using `keccak256`.

Smart Contract Implementation Details

The core logic resides in two contracts located in `contracts/contracts/`: 1. **`TenderProxy.sol`:** An ERC-1967 transparent upgradeable proxy contract holding all contract state and funds. Incoming calls are forwarded to the current implementation contract via low-level `delegatecall`. 2. **`OilTenderingV1.sol`:** The implementation contract containing all procurement business logic, access control modifiers, and state mutations. Inherits OpenZeppelin's `Initializable` and `UUPSUpgradeable`.

Roles and Access Control

- `multisigAdmin`: Can authorize UUPS logic upgrades (`_authorizeUpgrade`), register PSU entities (`registerPSU`), and configure global parameters (dispute bond amount, default minimum EMD).
- `registeredPSUs` (`mapping(address => bool)`): Authorized PSU officer addresses that can create tenders (`createTender`), cancel tenders (`cancelTender`), record technical evaluation scores (`setTechnicalScore`), and award contracts (`awardTender`).
- `registeredVendors` (`mapping(address => VendorInfo)`): Verified vendor addresses approved through the KYC process that can submit bids (`submitBid`), reveal quotes (`revealBid`), and

dispute awards (`disputeTender`).

- `keeperAddress`: Automated daemon authorized to trigger time-dependent lifecycle transitions (`openBids`, `finalizeTender`).
- `cvcAuditorAddress`: Central Vigilance Commission auditor authorized to adjudicate grievances (`resolveDispute`).

Core Data Structures

```
struct Tender {
    uint256 id;
    address psuAddress;
    string tenderIpfsHash;
    uint256 minEMDAmount;
    uint256 submissionDeadline;
    uint256 openingTime;
    uint256 revealDeadline;
    TenderStatus status;
    address winner;
    uint256 winningBidId;
    uint256 winningBidAmount;
    uint256 awardTimestamp;
}

struct Bid {
    uint256 id;
    uint256 tenderId;
    address vendorAddress;
    string encryptedBidIpfsHash;
    bytes32 bidCommitment;
    uint256 submissionTime;
    uint256 bidAmount;
    BidStatus status;
    bool isTechQualified;
    uint256 techScore;
    uint256 emdDeposited;
}

struct Dispute {
    uint256 id;
    uint256 tenderId;
    address disputingVendor;
    string reasonIpfsHash;
    uint256 timestamp;
    DisputeStatus status;
    string resolutionNotes;
    uint256 bondAmount;
}
```

Smart Contract Public Function Reference

Table 3 details all public and external functions implemented in `OilTenderingV1.sol`:

Function Signature	Access Guard	Payable?	Functional Purpose and Core Invariants
`initialize(admin, keeper, cvc, treasury, bond, emd)`	External	No	Initializes proxy storage. Sets administrative roles and parameters. Protected by `initializer` modifier.
`registerPSU(address _psuAddress)`	`onlyAdmin`	No	Authorizes an institutional PSU procurement address to create and manage tenders.
`registerVendor(vendor, gstinHash, panHash, kycHash)`	`onlyAdmin`	No	Records vendor KYC accreditation, storing cryptographic hashes of GSTIN and PAN.
`createTender(ipfsHash, subDead, openTime, revDead, minEMD)`	`onlyPSU`	No	Publishes a tender. Enforces now < subDead ≤ openTime < revDead.
`cancelTender(tenderId, reason)`	`onlyPSU`	No	Cancels an active tender before finalization. Calls `_creditRefunds()` to refund all EMDs.
`submitBid(tenderId, encryptedDocCid, commitment)`	`onlyVendor`	**Yes**	Accepts sealed bid. Enforces now < subDead, msg.value ≥ minEMD, and duplicate-bid prevention (`!hasBid`).
`openBids(tenderId)`	`onlyKeeper`	No	Transitions status to `UnderEvaluation`. Enforces now ≥ openingTime and status is `Created`.
`revealBid(tenderId, bidIndex, bidAmount, salt)`	`onlyVendor`	No	Unseals bid. Asserts keccak256(msg.sender, bidAmount, salt, tenderId) = = storedCommitment.
`setTechnicalScore(tenderId, bidIndex, qualified, score)`	`onlyPSU`	No	Records technical score (0–100) and qualification status for opened bids.
`awardTender(tenderId, winner, winningBidId, amount)`	`onlyPSU`	No	Enforces L1 award on-chain: iterates over bids,

Function Signature	Access Guard	Payable?	Functional Purpose and Core Invariants
			verifies winner is qualified, and asserts quote is strictly lowest. Starts 7-day standstill.
`disputeTender(tenderId, reasonIpfsHash)`	`onlyVendor`	**Yes**	Lodges appeal during 7-day standstill. Requires msg.value ≥ disputeBond. Transitions to `Disputed`.
`resolveDispute(tenderId, upheld, resolutionNotes)`	`onlyCVC`	No	Adjudicates dispute. If upheld: resets winner, status returns to `UnderEvaluation`, refunds bond. If dismissed: bond to treasury.
`resolveDisputeTimeout(tenderId)`	Public	No	Timeout failsafe: allows any caller to auto-resolve dispute if CVC is inactive for > 14 days.
`finalizeTender(tenderId)`	`onlyKeeper`	No	Finalizes tender when now ≥ awardTimestamp + 7 days. Invokes `_creditRefunds()`.
`withdraw()`	Public	No	Pull-payment pattern: transfers caller's `pendingWithdrawals` balance via reentrancy-safe low-level call.

Defensive Security Patterns Implemented

1. **Commit-Reveal Secrecy:** Bids are sealed on-chain using H = keccak256(msg.sender, bidAmount, salt, tenderId). Binding `msg.sender` and `tenderId` prevents front-running and cross-tender replay attacks.
2. **Reentrancy-Safe Pull Payments:** The contract never pushes Ether directly to external addresses during batch finalizations. Instead, `_creditRefunds()` credits `pendingWithdrawals[vendor]`. Vendors withdraw via `withdraw()`, which adheres strictly to the **Checks-Effects-Interactions** pattern (setting balance to 0 prior to `.call{value: amount}("")`).
3. **Algorithmic On-Chain L1 Enforcement:** During `awardTender()`, the contract checks all bids in storage. If the proposed winner does not match the lowest qualified bid, the transaction reverts with: `"Must award to lowest qualified bid (L1)"`.

4. **Duplicate Bid Blocking:** A storage mapping `hasBid[tenderId][vendorAddress]` blocks vendors from submitting multiple commitments for the same tender.
5. **Input Validation and Caps:** The backend rejects missing or malformed EMD values (eliminating legacy silent defaults) and caps submissions at a configurable `MAX_EMD_ETH` (default 1 ETH) to prevent accidental loss of funds.

Known Limitations

- **Single Backend Signing Key:** In the current demonstration prototype, the backend server relays gas transactions for technical scoring and awards using a single private key (`OFFICER_PRIVATE_KEY`). In production, each PSU must maintain an isolated signing HSM.
- **Deployer Role Overload:** In the prototype, the deployer wallet acts as multisig admin, PSU officer, and upgrade authority. In production, these roles must be partitioned across independent multi-signature contracts.
- **Keeper Dependency:** State transitions (`openBids` and `finalizeTender`) rely on the keeper bot running. If the bot is offline, tenders pause until the keeper restarts. (However, all functions remain callable manually by authorized accounts).
- **Testnet Visibility:** Because Sepolia is a public testnet, transaction metadata (bidder wallet addresses and timestamps) is publicly visible on Etherscan, though bid amounts remain concealed until reveal.
- **No Formal Verification:** While the codebase passes 73 automated tests with 100% success, it has not undergone formal mathematical verification or an external third-party security audit.

---

11. Sample Screenshots

The following 10 checkpoints illustrate the complete end-to-end user workflow across the NIRVAAN platform. Bounded visual image placeholders are provided below:

```
+-------------------------------------------------------------------------------
------------------+
|     [FIGURE     4     PLACEHOLDER:     LANDING     PAGE     &     TENDER     BROWSER]
|
|                      File:                      docs/screenshots/01_landing_page.png
|
|  Description:  Public  tender  discovery  portal  (https://nirvaan-flame.vercel.app/)
displaying        |
| active tenders across IOCL, BPCL, and ONGC, along with the top GIGW accessibility
toolbar.          |
+-------------------------------------------------------------------------------
------------------+
```

```
+-------------------------------------------------------------------------------
------------------+
|      [FIGURE      5      PLACEHOLDER:      OFFICER      AUTHENTICATION      PORTAL]
|
|                     File:                     docs/screenshots/02_officer_login.png
|
|  Description:  PSU  Officer  login  interface  (/officer/login)  showing  credentials
authentication,      |
|   bcrypt   password   verification,   and   per-PSU   tenant   organization   selection.
|
+-------------------------------------------------------------------------------
------------------+
```

```
+-------------------------------------------------------------------------------
------------------+
|       [FIGURE       6       PLACEHOLDER:       TENDER       CREATION       INTERFACE]
|
|                    File:                    docs/screenshots/03_tender_creation.png
|
| Description: Tender creation form (/officer/create-tender) showing fields for NIT
code, category,  |
| display budget (INR), EMD (ETH), submission/opening/reveal deadlines, and IPFS
specification upload.|
+-------------------------------------------------------------------------------
------------------+
```

```
+-------------------------------------------------------------------------------
------------------+
|     [FIGURE     7     PLACEHOLDER:     VENDOR     KYC     ONBOARDING     APPLICATION]
|
|                File:                docs/screenshots/04_vendor_kyc_application.png
|
| Description: Vendor registration form (/vendor/register) displaying company name,
GSTIN, PAN, and  |
|     the     MetaMask     wallet     challenge-response     nonce     signing     modal.
|
+-------------------------------------------------------------------------------
------------------+
```

```
+-------------------------------------------------------------------------------
------------------+
|     [FIGURE     8     PLACEHOLDER:     OFFICER     KYC     APPROVAL     DASHBOARD]
|
|                 File:                 docs/screenshots/05_officer_kyc_approval.png
|
|  Description:  Officer  vendor  management  dashboard  (/officer/vendors)  displaying
submitted           |
|    applications    with    on-chain    Approve    and    Reject    action    triggers.
|
+-------------------------------------------------------------------------------
------------------+
```

```
+-------------------------------------------------------------------------------
------------------+
|     [FIGURE     9     PLACEHOLDER:     SEALED     BID     SUBMISSION     FORM]
|
|                File:                docs/screenshots/06_sealed_bid_submission.png
|
|  Description:  Sealed  bid  submission  interface  (/tenders/:id/bid)  showing  EMD
prefilled from the     |
| tender's on-chain minEMDAmount, client-side salt generation, and the "Download
Backup JSON" prompt. |
+-------------------------------------------------------------------------------
------------------+
```

```
+-------------------------------------------------------------------------------
------------------+
|     [FIGURE     10     PLACEHOLDER:     COMMERCIAL     BID     REVEAL     INTERFACE]
|
|                   File:                   docs/screenshots/07_bid_reveal_page.png
|
|  Description:  Commercial  reveal  modal  (/tenders/:id)  showing  unsealing  of  the
stored 32-byte salt    |
|  and  bid  amount  during  the  active  reveal  window,  updating  status  to  Opened.
|
+-------------------------------------------------------------------------------
------------------+
```

```
+-------------------------------------------------------------------------------
------------------+
|     [FIGURE     11     PLACEHOLDER:     TECHNICAL     PROPOSAL     EVALUATION     VIEW]
|
|                 File:                 docs/screenshots/08_bid_evaluation_page.png
|
|  Description:  Technical  evaluation  dashboard  (/tenders/:id/evaluate)  showing
opened bids, technical|
| scores, qualification status, and the highlighted "Lowest Qualified (L1)" badge.
|
+-------------------------------------------------------------------------------
------------------+
```

```
+-------------------------------------------------------------------------------
------------------+
|     [FIGURE     12     PLACEHOLDER:     CONTRACT     AWARD     &     CUSTODY     TIMELINE]
|
|                File:                docs/screenshots/09_award_custody_timeline.png
|
| Description: Tender detail view (/tenders/:id) displaying the dynamic chain-of-
custody timeline     |
| with real transaction hashes and block numbers hyperlinked to Sepolia Etherscan.
|
+-------------------------------------------------------------------------------
------------------+
```

```
+-------------------------------------------------------------------------------
------------------+
|    [FIGURE    13    PLACEHOLDER:    ETHERSCAN    ON-CHAIN    TRANSACTION    RECEIPT]
|
|                File:                docs/screenshots/10_etherscan_transaction.png
|
|  Description:  Public  Sepolia  Etherscan  block  explorer  receipt  verifying  an
executed tender          |
|           lifecycle           transaction           (contract           address:
0xe9649E82dC4fEdf86dFbE5120a3fE79BDde88663).             |
+-------------------------------------------------------------------------------
------------------+
```

(Note: The student should capture the 10 screen captures listed above, place them into `docs/screenshots/`, and insert them into the Word document or PDF before final submission.)

---

12. Conclusion

The NIRVAAN project designed, implemented, and empirically verified a decentralized e-tendering platform tailored to the high-value procurement challenges of Indian oil and gas PSUs. By synthesizing an upgradeable Solidity smart contract architecture (`UUPS` / `ERC-1967`), a non-interactive commit-reveal bidding scheme, on-chain L1 consensus enforcement, an automated pull-payment escrow mechanism, and a statutory 7-day standstill governance window with independent CVC arbitration, the platform demonstrates that public procurement can transition from subjective administrative trust to verifiable algorithmic consensus.

Real Test Execution Results

The platform's correctness and security invariants were verified through three automated test suites: - **Hardhat Smart Contract Test Suite: 13 / 13 passing** (100% pass rate). Validates proxy deployment, role-based access control, deadline guards, duplicate-bid prevention, commit-reveal verification, L1 award enforcement, dispute timeouts, and cancellation refunds. - **Backend Unit & Integration Test Suite: 24 / 24 passing** (100% pass rate). Validates indexer fallback parsing for Alchemy 10-block query limits, batch halving, ETH decimal string parsing, GSTIN/PAN regex validation, and vendor database state reconstruction. - **End-to-End System Smoke Test: 36 / 36 passing** (100% pass rate). Validates security guards, unauthenticated route rejections, challenge nonce verification, officer login and password changes, vendor KYC application and approval, tender creation, and on-chain award transitions. - **Total Passing Automated Tests: 73 / 73 passing clean**.

Lessons Learned

- **Separation of Consensus and Content:** Blockchains are optimized for state transitions, financial accounting, and cryptographic hashes. Storing large documents directly on-chain is cost-prohibitive; off-chain decentralized storage (IPFS) referenced via CIDs provides the optimal balance of integrity and gas efficiency.
- **RPC Constraints on Public Testnets:** Public RPC providers (such as Alchemy free tier) enforce strict query limits (e.g., 10 blocks per `eth_getLogs` call). An adaptive indexer capable of

automatically halving batch ranges and retrying with exponential backoff is essential for backend resilience.

- **Defensive Financial Patterns:** Smart contracts must never push funds to external addresses during state transitions. Adhering strictly to the pull-payment pattern protects the contract against denial-of-service and reentrancy exploits.

Future Work

1. **Multi-Signature Governance:** Transitioning the platform administrative role from a single wallet to a multi-signature contract (e.g., Safe) requiring unanimous approvals from participating PSUs.
2. **Dedicated Per-PSU Signing Relays:** Implementing dedicated Hardware Security Modules (HSMs) for each individual PSU rather than sharing a single backend signing key.
3. **Decentralized Keeper Network:** Integrating decentralized automation networks (such as Chainlink Automation or Gelato Network) to remove the reliance on a centralized keeper service.
4. **Zero-Knowledge Bid Commitments:** Implementing zk-SNARK circuits to enable bidders to prove that their sealed quote complies with statutory budget caps without revealing the quote value during the commitment phase.
5. **Production Migration:** Deploying the architecture to a dedicated private consortium network running Hyperledger Besu (QBFT consensus) to eliminate public gas fees, provide sub-second finality, and ensure regulatory data sovereignty.